Weekly Progress Report

EGN4950C Senior Design, Group 16

Kheiven D'Haiti | Jorge Sanchez | Ashleyn Montano | Riley Roberts | Victor De Souza Teixeira

Report Date: August 31, 2026. Period: August 31 to September 6, 2026.

EGN4950C Senior Design

Weekly Progress Report

Project Title: Open Source Selective Video Compression for Static Surveillance Cameras
Technical Sponsor: Defense Innovation Unit (DIU) / NIWC Pacific, Cody Hayashi **Group:** 16
Report Date: August 31, 2026 **Report Period:** August 31, 2026 to September 6, 2026 **Team Members:** Kheiven D'Haiti (CS, AI Minor) | Jorge Sanchez (CS) | Ashleyn Montano (CS) | Riley Roberts (CS) | Victor De Souza Teixeira (CS, Cybersecurity)

Part 1: Team Section

All team members have detailed tasks listed on Teams Planner: YES. A new planner was built this period covering all fourteen weeks of the semester, with weeks 1 through 6 broken down into individual assigned tasks and weeks 7 through 14 outlined by theme. Source document: `docs/PLANNER-FALL-2026.md`.

1. Team Meeting

Did the team meet this period? [FILL IN: date, time, and who attended. If the team did not meet, say so and describe how coordination happened instead.]

2. Sponsor / Advisor Meeting

Did the team meet with the sponsor this period? [FILL IN. The last formal sponsor check-in on record is April 8, covered in spring Report 2. State when the next one is planned.]

3. Team Progress This Period

This was the first week of the fall semester, and the work was review and planning rather than construction.

The team reviewed what was built last semester and over the summer break. Between May 1 and August 18, development continued on the repository: 158 commits, 401 files changed, 62,174 lines added and 11,278 removed, all pushed. The desktop application gained a Windows

installer, a video library, an auto-compress watch-folder service, disk-budget retention, and a sixteen-item security audit. Compression improved measurably through VMAF-targeted rate control and encoder-level ROI. A new Android companion application was built from nothing to version 0.9.0-beta. The Python test suite grew from 274 passing tests to 1,651, and the Flask API from 48 routes to 87 across 22 blueprints.

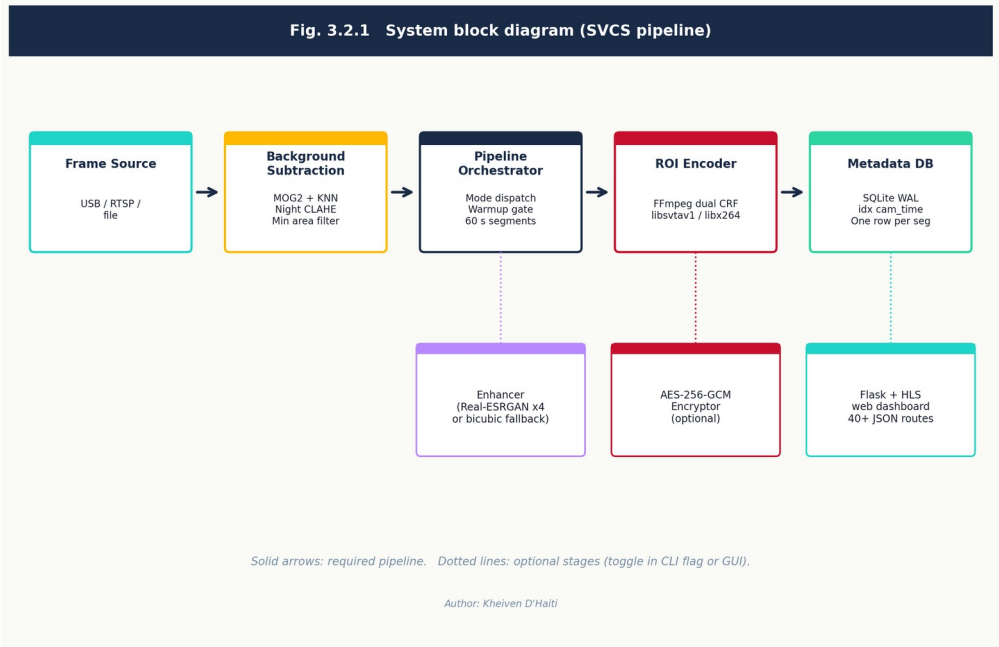

Figure 1. SVCS system architecture. Footage enters from a camera, a live RTSP stream, or a watched drop folder, passes through background subtraction and the YOLO classification gate, is filtered by compression mode, then encoded with region-of-interest awareness and indexed in SQLite.

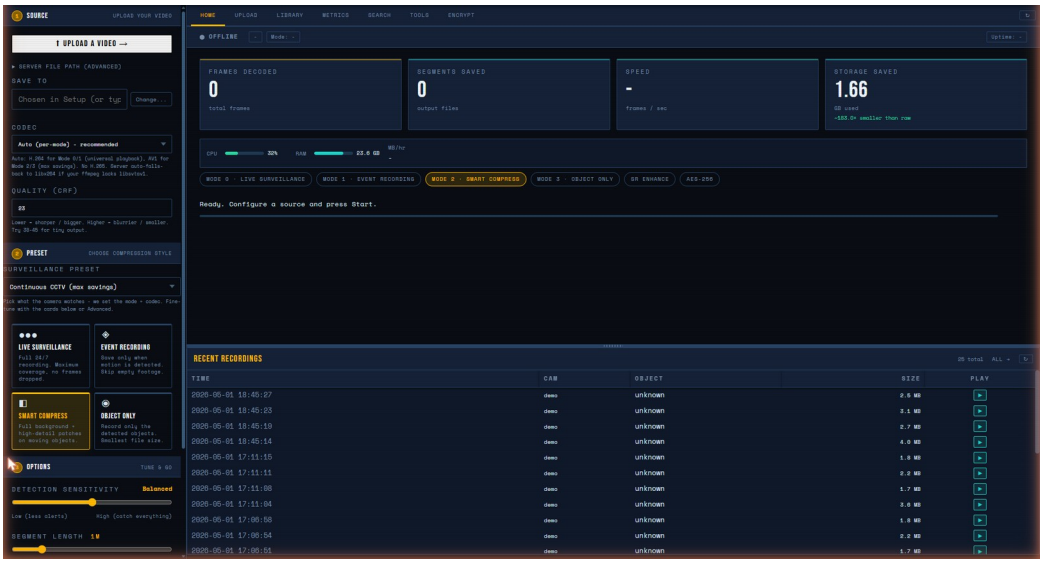

Figure 2. The desktop dashboard. The topbar tabs (UPLOAD, LIBRARY, AUTO-COMPRESS, METRICS, SEARCH, TOOLS, ENCRYPT) were all added over the summer. In April this was a single page.

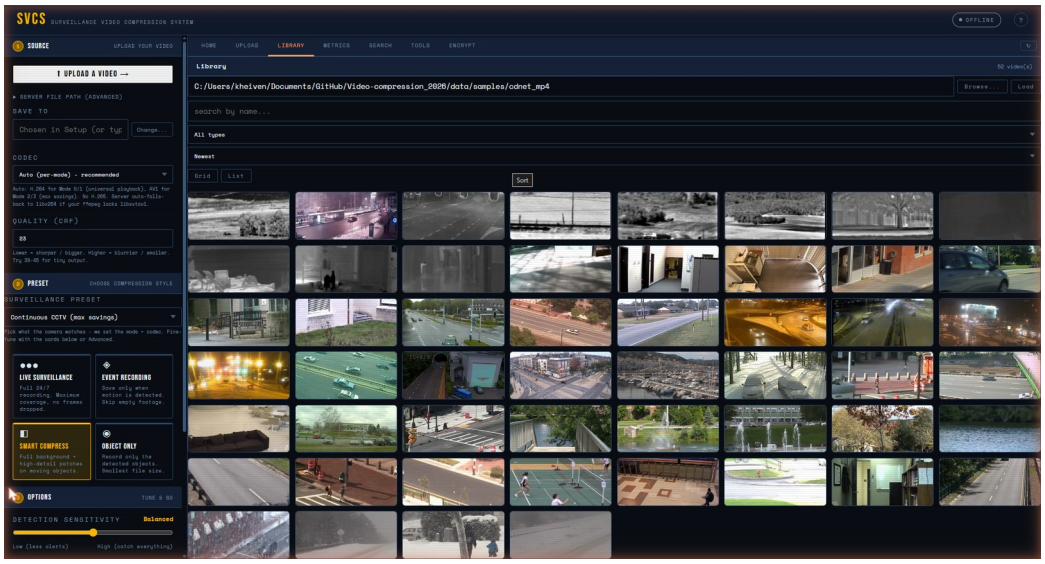

Figure 3. The video library added in June, with folder picker, search, extension filter, and sort. Listing 5,000 files took 2,153 ms before the caching work and 3.3 ms after.

The consequence for planning is that the semester's remaining work is not "build the project." The project builds, installs, and runs. What remains is finishing the mobile port, exposing the zone and event system on the desktop where only the phone can currently reach it, and hardening the whole thing into something the sponsor can deploy.

Kheiven built a new semester planner on that basis. It assigns work by the subsystem each member already owned in the spring, so nobody restarts from zero: Jorge keeps ingest and streaming, Ashleyn keeps metadata and queries, Riley keeps compression modes, and Victor keeps security and encryption. The plan is organized into four phases with a milestone each, has two tasks per member per week for the first six weeks, and records dependencies and risks.

4. Challenges and Blockers

- **The mobile application does not persist its pairing.** After a successful connection test, restarting the app returns to an older credential. This was isolated by pointing the app at a logging HTTP listener, which showed it sending a token ending `s6WlSc` while the token that had just been validated ended `62jf5Y`. The server agrees: the stored token's `last_used_at` timestamp advances at the moment of the connection test and never again. This blocks the phone's notification settings panel and is scheduled as tasks 4.1 and 4.2.
- **The mobile application has almost no automated tests.** One JVM test file exists, which is why every mobile change over the summer had to be verified by hand on a physical device. That does not scale to five people working in parallel, so building the test harness is scheduled first, in week 3, ahead of the pairing fix that depends on it.

- **A test in the suite was destroying real application state.** `tests/security/test_csrf.py` posts to real state-changing routes to prove the CSRF guard allows same-origin requests. One of those routes is a factory reset, so every full test run deleted the stored device tokens and unpaired every phone. It presented as a broken phone rather than a broken test. Fixed in commit `f44f51e`.
- **Four of five members were away from the repository over the break** and the areas they own changed underneath them. This is the main risk to the first three weeks, and it is why the changes document and the ramp-up tasks in week 1 exist.

5. Summary Tables

Previous report period: April 15 to April 26, 2026 (spring Report 3)

Member	Tasks completed	Not completed	Tasks for next period
Kheiven D'Haiti	5	0	4
Jorge Sanchez	2	0	2
Ashleyn Montano	1	0	2
Riley Roberts	2	0	2
Victor De Souza Teixeira	2	0	2

Table 1. Previous report summary, spring Report 3.

Current report period: August 31 to September 6, 2026

Member	Tasks completed	Not completed	Tasks for next period
Kheiven D'Haiti	2	0	3
Jorge Sanchez	2	0	2
Ashleyn Montano	2	0	2
Riley Roberts	2	0	2
Victor De Souza Teixeira	2	0	2

Table 2. Current report summary.

Part 2: Individual Sections

Task numbers match the MS Teams Planner and `docs/PLANNER-FALL-2026.md`.

Kheiven D'Haiti, CS Major, AI Minor

Report Date: August 31, 2026

Tasks completed this reporting period

Task 1.1: Review the summer build and audit repository state (completed September 5)

Description: Development continued on the repository through the summer break, and the team needed an accurate picture of what the project actually is now before planning around it. I went back through the commit history from May 1 to August 18 and reconstructed what changed, round by round, then verified that nothing was stranded on my local machine.

Implementation and results: 158 commits over the break; 401 files changed, 62,174 lines added and 11,278 removed. I grouped them into the rounds they were built in: the May refactor that split `gui/app.py` into a service and blueprint layer and got the suite green; the June installer, preset, and library work; the June security audit covering SEC-001 through SEC-016; the July compression round that added encoder-level ROI, long GOP, NVENC, denoise, and VMAF measurement, plus disk-budget retention; the July mobile milestones M0 through M3; and the August work on zones, behavior events, and the mobile app through version 0.9.0-beta.

I then checked every branch against its remote. All six tracked branches report zero unpushed commits, and `mobile` and `app` both sit at commit `faeebd1` matching their remotes. Three consolidation documents written on August 17 are present on disk but not yet committed: `docs/RESEARCH.md` at 1,540 lines, `docs/SECURITY.md` at 405 lines, and `docs/archive/PROJECT-HISTORY.md` at 872 lines. Committing those is scheduled as task 2.2.

Measured change in project state:

Metric	April 2026	September 2026
Python tests passing	274	1,651
Test files	about 20	96
Flask routes	48	87 across 22 blueprints
Desktop distribution	run from source	signed installer, 217 MB
Android application	did not exist	0.9.0-beta, 4.5 MB APK
Library listing at 5,000 files	2,153 ms	3.3 ms

Table 3. Project state before and after the summer.

Outcome: An accurate baseline to plan from, and confirmation that no work is stranded locally. This directly enabled task 1.2, because a plan built on a wrong picture of the codebase would have assigned work that is already done.

Evidence: See Figure 4.

```
PS C:\Users\kheiven\Documents\GitHub\Video-compression_2026> git branch -vv
  app     faeebd1 [origin/app]     docs(r7): handoff spec for Claude Code
  dev     9d95716 [origin/dev]     feat: sponsor handoff prep
  kdev    a300cdb [origin/kdev]    chore: relicense as AGPL-3.0 dual-license
  main    c2ba6e1 [origin/main]    Merge dev to main for sponsor handoff
* mobile  faeebd1 [origin/mobile]  docs(r7): handoff spec for Claude Code
  premium 10cbda9 [origin/premium] refactor: audit fixes

PS> foreach branch: git rev-list --left-right --count <branch>...<upstream>

  app     -> origin/app      : ahead 0, behind 0
  dev     -> origin/dev      : ahead 0, behind 0
  kdev    -> origin/kdev     : ahead 0, behind 0
  main    -> origin/main     : ahead 0, behind 0
  mobile  -> origin/mobile   : ahead 0, behind 0
  premium -> origin/premium  : ahead 0, behind 0

Result: zero unpushed commits on all six tracked branches.
mobile and app both at faeebd1, matching their remotes.
```

Figure 4. Branch-versus-remote comparison. All six tracked branches report zero unpushed commits, with `mobile` and `app` both at commit `faeebd1`.

Task 1.2: Build the fall semester planner (completed September 6)

Description: The spring planner ended with the capstone demo and does not describe this semester. I built a new one covering all fourteen weeks, from August 31 through December 6, and populated the MS Teams Planner from it.

Implementation: I organized the semester into four phases, each ending in a milestone. Phase A, weeks 1 to 3, is planning, the revised proposal, and building a mobile test harness. Phase B, weeks 4 to 7, is the desktop zone and event interface plus uploads that survive the app being killed. Phase C, weeks 8 to 11, is native push, semantic search, and measured compression results. Phase D, weeks 12 to 14, is regression, the 1.0 release, the final report, and the demo.

Assignment follows the subsystems each member already owned in the spring, so that nobody has to learn a new area to contribute. Jorge keeps ingest and streaming and takes the upload worker and the multi-camera HLS registry. Ashleyn keeps metadata and queries and takes the event panel and semantic search. Riley keeps the compression modes and takes the zone editor and the measurement of what zone masking actually saves. Victor keeps security and takes the webhook emitter and a review of the mobile credential path.

Weeks 1 through 6 are broken into individual tasks sized to a few hours each, two per member per week, each with an owner, dates, and a checkable outcome. Weeks 7 through 14 are outlined by theme and will be detailed on a rolling four-week basis as the planner requirement specifies. The plan also records eight task dependencies and a five-item risk register.

The largest sequencing decision was to put the mobile test harness in week 3, before the pairing fix in week 4, even though the pairing defect is the more urgent problem. Diagnosing it on a minified release build with no tests is what consumed most of a session in August. Building the harness first is slower for one week and faster for the eleven after it.

Outcome: `docs/PLANNER-FALL-2026.md` and a companion CSV that imports into MS Teams Planner. Every member has assigned, dated tasks for at least the next four weeks, which satisfies the planner requirement and gives everyone something concrete to write about in next week's report.

Phase	Weeks	Dates	Goal	Milestone
A. Foundations	1 to 3	Aug 31 to Sep 20	Planning, revised proposal, mobile test harness, fix the pairing defect	M1: mobile changes verifiable without a human holding the phone
B. Feature completion	4 to 7	Sep 21 to Oct 18	Desktop zone and event interface, uploads that survive app death, multi-camera streaming	M2: desktop and mobile parity on zones and events
C. Advanced work	8 to 11	Oct 19 to Nov 15	Native push, semantic search, measured compression results	M3: all R5 and R6 tracks closed
D. Hardening and delivery	12 to 14	Nov 16 to Dec 6	Regression, 1.0 release, final report, sponsor demo	M4: release and final demo

Table 4. Semester phases and milestones, from the new planner.

Evidence: The phase and milestone breakdown is reproduced as Table 4.

Planned tasks for the coming period (September 7 to September 13)

Task 2.1: Write the summer changes document for the team The team cannot act on 158 commit messages. I am writing a round-by-round account of every change from May through August covering what changed, why, and what it means for each member's subsystem, with a before-and-after metrics table, the current list of known defects, and a reading guide mapping each person's area to the files they should open first. Expected outcome: `docs/CHANGES-SUMMER-2026.md` committed and pushed, reducing team ramp-up from reading a commit log to reading one document. It also becomes source material for the final report, since it records the reasoning behind decisions and not only the outcomes.

Task 2.2: Commit the stranded documentation and re-verify push status `docs/RESEARCH.md`, `docs/SECURITY.md`, and `docs/archive/PROJECT-HISTORY.md`

total roughly 2,800 lines and exist only on my machine. Commit them and confirm all branches clean. Expected outcome: no project documentation exists in only one place.

Task 2.3: Revised proposal, system design section and functional diagrams The Module 1 revised proposal is due September 13. The system design section still describes a desktop-only pipeline. I will redraw the architecture diagram to include the mobile client, the device token boundary, and the push path, and mark the revisions in red as the assignment requires. Main challenge is that the architecture changed enough over the summer that this is closer to a rewrite than an edit.

Jorge Sanchez, CS Major

Report Date: August 31, 2026

Tasks completed this reporting period

Task 1.3: Review my spring contributions and the current state of the ingest subsystem (completed September 5)

Description: Last semester I built the ingest side of the pipeline. The watchfolder daemon in `src/utils/watchfolder.py` polls a drop folder every five seconds, detects new video by extension, waits for the file to stop growing before ingesting it, and writes a sentinel beside each processed file so a restart does not double-process. I also built `MultiFrameSource` in `src/utils/multi_source.py`, which manages N parallel RTSP streams through reader daemon threads with a two-frame ring buffer and a five second stall timeout. Both shipped in PR #13 with 22 tests. Earlier in the semester I built the streaming ROI encoder and ran the MOG2 versus KNN benchmark across 46 CDnet scenes and 30 parameter combinations.

This period I went back through that code and looked at what the summer changed underneath it. The relevant change is R4 Phase 6, which added universal multi-vendor format support through an FFmpeg decode fallback, so vendor formats like `.dav`, `.g64`, and `.mxf` now ingest. That sits directly on top of my watchfolder extension detection, and there is a follow-up commit fixing MPEG-TS frame drops and making the ffmpeg reader stall-proof, which overlaps my stall detection logic.

Outcome: I know what changed in my area and where the new decode fallback meets my existing code. No conflicts found, but the stall handling now exists in two places and should be reconciled.

Task 1.4: Team review session on the summer build and fall ownership (completed September 6)

Description: The team went through what was built over the break and decided who takes what this semester. The project is further along than it was in April, so the remaining work is finishing the mobile port and exposing features the phone can reach but the desktop cannot.

Outcome: I am keeping ingest and streaming. My two areas for the fall are making phone uploads survive the app being killed, and replacing the single global HLS stream slot with a per-camera registry so more than one person can watch at once.

Planned tasks for the coming period (September 7 to September 13)

Task 2.4: Revised proposal, requirements list including mobile requirements The requirements section of our proposal was written before the phone client existed, so it says nothing about it. I am adding requirements covering the mobile client: what it must do, what it must not do, and the constraints it operates under, such as running on a private network over plain HTTP. New text goes in red per the assignment.

Task 3.3 and 3.4: WorkManager upload design and protocol audit The chunked resumable upload shipped in August, but the transfer runs in `viewModelScope`, so Android killing the app kills the upload. Before writing anything I need to confirm that `/api/upload/status` returns a byte offset a background worker can actually resume from, because if it does not, the worker is pointless and restarts at zero anyway. Then I will design the `CoroutineWorker` that replaces the current transfer.

Ashleyn Montano, CS Major

Report Date: August 31, 2026

Tasks completed this reporting period

Task 1.5: Review my spring contributions and the current state of the metadata subsystem (completed September 5)

Description: Last semester I owned the metadata database and the query interface. I extended `query_by_type()` in `src/utils/db.py` to accept either a single string or a list of object types, using parameterized IN placeholders rather than string interpolation so SQL injection safety is maintained, and I added a `min_roi_count` filter for dropping low-confidence detections. I rewrote the CLI in `src/utils/db_query.py` at the same time so `--type` accepts multiple values, `--min-roi` filters by ROI count, and `--last-hours` generates timezone-aware UTC timestamps automatically.

This period I reviewed what the summer added on top of that. Two things matter for my area. First, `/api/nl_search` was added in August, a natural-language search endpoint that sits alongside my structured query interface rather than replacing it. Second, the behavior event system now writes events to an `events.jsonl` file next to the footage and exposes them through `/api/events/recent`, which is a second data source the query surface does not know about yet.

Outcome: I understand where the new search and event data lives and how it relates to the SQLite metadata I already query. The gap I found is that events are visible from the phone but not from the desktop at all.

Task 1.6: Team review session on the summer build and fall ownership (completed September 6)

Description: The team reviewed the summer work and assigned fall ownership.

Outcome: I am keeping metadata and queries. My fall work is building the desktop EVENTS panel so the behavior events are visible outside the phone, and then the semantic search task that has been open since R5.

Planned tasks for the coming period (September 7 to September 13)

Task 2.5: Revised proposal, literature survey and reference list The literature survey needs updating and the citations need to be brought to one consistent format. Several research documents were written over the summer that have sources not yet reflected in the proposal, so I am pulling those in.

Task 3.5 and 3.6: Desktop EVENTS panel The behavior event system fires on line crossings and loitering, but nothing on the desktop displays it, so an operator sitting at the dashboard cannot see alerts the phone receives. I am building a collapsible panel on the TOOLS tab showing events newest first with kind, camera, headline, and wall time, refreshing every ten seconds while the tab is visible and stopping when it is hidden. The empty state has to tell the operator to draw zones and run a compress rather than just saying no data, because an empty panel with no explanation reads as broken. The specification is in `docs/DESKTOP-ZONES-EVENTS-PLAN.md` section D1.

Riley Roberts, CS Major

Report Date: August 31, 2026

Tasks completed this reporting period

Task 1.7: Review my spring contributions and the current state of the compression modes (completed September 5)

Description: Last semester I implemented Mode 2 and Mode 3 and merged them in PR #11. Mode 2 captures one clean background frame during warmup, or falls back to the last warmup frame if no fully static frame appears, then composites per-frame object patches over it, so the segment encodes only moving content on top of a frozen reference background. Mode 3 passes `object_only=True` to the ROI encoder, which blacks out every pixel outside a detected bounding box. I also repaired the mode dispatch in `modes.py` after the streaming encoder refactor had silently made every mode fall back to Mode 0 behavior.

This period I reviewed what the summer changed. R4 Phase 2 is the significant one: the encoder now supports long GOP, capped CRF, NVENC hardware encoding, denoise, and encoder-level ROI, which means the encoder is told where the interesting pixels are instead of us blacking out everything else ourselves. That overlaps Mode 3 directly, since Mode 3 does the blacking out manually. R5 5.1 then added VMAF-targeted rate control, which searches for the

smallest file that still meets a quality floor rather than using a fixed CRF. Mode 3 was also rewritten in May to produce per-object videos instead of blackout-in-full-frame.

Outcome: I know where my modes now sit relative to the encoder work. The open question is whether Mode 3's manual blackout is still the right approach now that the encoder accepts ROI hints directly, which is worth measuring.

Task 1.8: Team review session on the summer build and fall ownership (completed September 6)

Description: The team reviewed the summer work and assigned fall ownership.

Outcome: I am keeping the compression side. My fall work is the desktop zone editor, since exclude zones feed directly into the encoder ROI decisions I already own, and measuring what zone masking actually saves in file size.

Planned tasks for the coming period (September 7 to September 13)

Task 2.6: Revised proposal, Gantt chart I am building the Gantt from the new semester planner: fourteen weeks, every task with an owner and a duration, the dependencies between them, and the four phase milestones marked.

Task 3.7 and 3.8: Zone editor groundwork The zone and line APIs exist and work from the phone, but the desktop has no way to draw them, which means the operator has to use a phone to configure a desktop application. Before the drawing interface can exist there has to be something to draw on, so I am adding `GET /api/zones/frame`, which returns the newest thumbnail for a camera as a JPEG or a black 16:9 placeholder when no thumbnail exists. It reuses the existing thumbnail pipeline so there is no new ffmpeg surface. Adding a route means updating all three route guards in the same commit. Then I draft the canvas layout mirroring the phone editor. Specification is in `docs/DESKTOP-ZONES-EVENTS-PLAN.md` section D2.

Victor De Souza Teixeira, CS Major, Cybersecurity

Report Date: August 31, 2026

Tasks completed this reporting period

Task 1.9: Review my spring contributions and the current state of the security subsystem (completed September 5)

Description: Last semester I upgraded `src/utils/encryption.py` from AES-256-CBC to AES-256-GCM in PR #12. CBC has no authentication tag, so a bit-flip attack could silently corrupt a stored video segment with no detection. GCM adds a 128-bit authentication tag, so any modification to the ciphertext raises `InvalidTag` before any plaintext is returned. The file format stores the nonce, salt, auth tag, and ciphertext in sequence, and the public API did not change. I also built the enhancement module with CPU, CUDA, and Apple MPS support and a `detect_gpu()` capability report.

This period I reviewed the summer security work. The June audit closed SEC-001 through SEC-016, and several items are adjacent to my area: a same-origin CSRF guard on every state-changing request, media and library file serving confined to the operator's own folders, encrypt path confinement, and an SSRF input guard on `input_source`. In August a second SSRF guard was written for the push feature with deliberately inverted policy, allowing loopback and private addresses because that is where a self-hosted notification server lives, while refusing cloud metadata endpoints and following no redirects.

Outcome: I know which guards exist now and what each one is for. The important detail for my fall work is that there are now two SSRF guards with opposite policies for different purposes, and any third network-egress feature must reuse one of them rather than invent a third.

Task 1.10: Team review session on the summer build and fall ownership (completed September 6)

Description: The team reviewed the summer work and assigned fall ownership.

Outcome: I am keeping security. My fall work is the webhook event emitter, which is a network egress feature and therefore a security problem first, and a review of the mobile credential storage path, which is currently returning a stale token after a restart.

Planned tasks for the coming period (September 7 to September 13)

Task 2.7: Revised proposal, organizational chart and completed task list The proposal has no organizational chart and the assignment requires one naming a team leader or project coordinator. I am building it along with the list of tasks and subtasks completed to date.

Task 3.9 and 3.10: Webhook emitter specification and test harness The webhook posts behavior events as JSON to an operator-configured URL, which means the server fetches a URL a user supplied, which is the classic server-side request forgery shape. The correct move is not to write a third guard but to reuse `push_notify.is_safe_push_url`, which already implements the right policy: allow loopback and RFC1918 because that is where a self-hosted receiver lives, and refuse non-http schemes, credentials embedded in the URL, cloud metadata hosts and addresses, hostnames whose DNS answer resolves into a blocked range, IPv4-mapped IPv6 forms of the same, and redirects. I am writing the specification and the socket-server test harness first, in the style of `tests/test_push_notify.py`, so the tests exist before the code does.
